ĐẠI HỌC THÁI NGUYÊN

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO THỰC HÀNH

MÔN: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

ĐỀ TÀI 29

TRIỂN KHAI WEBSITE CÔNG THỨC NẤU ĂN

KẾT HỢP NGINX, POSTGRESQL, PROMETHEUS, GRAFANA, LOKI VÀ PROMTAIL

Thông tin	**Nội dung**
Sinh viên	Lê Văn Huy
Mã sinh viên	DTC245200489
Lớp	CNTT K23A
Giảng viên hướng dẫn	Nguyễn Chuyên Anh
Đơn vị	Trường Đại học Công nghệ Thông tin và Truyền thông – Đại học Thái Nguyên

Thái Nguyên – 2026

LỜI NÓI ĐẦU

Trong quá trình phát triển phần mềm, việc xây dựng được ứng dụng mới chỉ là một phần công việc. Ứng dụng cần được triển khai nhất quán, có cơ sở dữ liệu, được bảo vệ ở mức cơ bản và có cơ chế theo dõi trạng thái hoạt động. Docker Compose giúp mô tả các thành phần dưới dạng cấu hình có thể tái lập, trong khi Nginx, Prometheus, Grafana, Loki và Promtail hỗ trợ truy cập, giám sát và tập trung nhật ký.

Báo cáo trình bày quá trình xây dựng website công thức nấu ăn mang tên Bếp Nhà và triển khai các dịch vụ phụ trợ. Nội dung được tổ chức theo các tiêu chí của đề thực hành: quản lý mã nguồn, ứng dụng và cơ sở dữ liệu, reverse proxy, giám sát metrics, tập trung log và hardening.

Do kết quả thực nghiệm phụ thuộc môi trường Docker trên máy thực tế, báo cáo cung cấp quy trình kiểm tra và các vị trí cần chèn ảnh chụp màn hình sau khi chạy. Sinh viên cần chạy hệ thống, lưu ảnh bằng chứng thực tế và cập nhật các kết quả đo được trước khi in/nộp.

MỤC LỤC

Mục lục tự động có thể được cập nhật trong Microsoft Word: nhấn Ctrl+A, sau đó nhấn F9 và chọn Update entire table. Nếu Word chưa hiển thị mục lục, vào References → Table of Contents.

DANH MỤC HÌNH ẢNH

- Hình 1.1. Kiến trúc tổng thể các dịch vụ (chèn ảnh sơ đồ tự vẽ hoặc ảnh Docker Compose).
- Hình 2.1. Giao diện trang chủ website Bếp Nhà.
- Hình 2.2. Trang chi tiết công thức nấu ăn.
- Hình 2.3. Biểu mẫu thêm công thức mới.
- Hình 3.1. Các container ở trạng thái Up trong Docker Compose.
- Hình 3.2. Cấu hình server Nginx và website truy cập qua cổng 8080.
- Hình 3.3. pgAdmin kết nối tới PostgreSQL và bảng recipe.
- Hình 3.4. Prometheus hiển thị target recipe-web ở trạng thái UP.
- Hình 3.5. Dashboard Recipe Website Overview trong Grafana.
- Hình 3.6. Truy vấn log của container trên Grafana Explore/Loki.

DANH MỤC BẢNG

- Bảng 1.1. Thành phần và vai trò.
- Bảng 2.1. Các tuyến chức năng của website.

- Bảng 3.1. Kịch bản kiểm thử và kết quả thực nghiệm.
- Bảng 4.1. Đối chiếu tiêu chí chấm điểm.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Website công thức nấu ăn là một bài toán gần gũi, có dữ liệu rõ ràng và dễ kiểm thử. Đề tài phù hợp để minh họa chuỗi triển khai ứng dụng web gồm giao diện, xử lý nghiệp vụ, cơ sở dữ liệu, reverse proxy, giám sát chỉ số và tập trung log.

1.2. Mục tiêu

- Xây dựng website xem, tìm kiếm và thêm công thức nấu ăn.
- Đóng gói ứng dụng và dịch vụ bằng Docker Compose để khởi chạy đồng bộ.
- Sử dụng PostgreSQL lưu dữ liệu và pgAdmin quản trị cơ sở dữ liệu.
- Cấu hình Nginx làm reverse proxy, bổ sung các HTTP security headers cơ bản.
- Dùng Prometheus thu thập metrics và Grafana trực quan hóa.
- Dùng Loki và Promtail tập trung log, hỗ trợ truy vấn LogQL.
- Áp dụng hardening cơ bản: user không phải root, giới hạn capability, read-only filesystem cho ứng dụng, mạng backend nội bộ và không công khai cổng PostgreSQL.

1.3. Phạm vi và giới hạn

Hệ thống được thiết kế cho môi trường học tập trên máy cá nhân. Cấu hình chưa thay thế quy trình vận hành production: chưa cấu hình TLS với chứng chỉ hợp lệ, chưa triển khai sao lưu tự động, quản lý secret chuyên dụng hoặc xác thực người dùng trên website. Không nên mở trực tiếp các cổng giám sát ra Internet.

1.4. Công nghệ sử dụng

Thành phần	**Vai trò**
Python / Flask / Gunicorn	Xử lý yêu cầu HTTP và cung cấp trang web.
PostgreSQL 16	Lưu trữ công thức nấu ăn.
pgAdmin	Quản trị và kiểm tra dữ liệu DB.
Docker Compose	Khai báo, kết nối và vận hành nhiều container.
Nginx	Reverse proxy, giới hạn kích thước request và security headers.
Prometheus	Thu thập metrics từ ứng dụng và chính Prometheus.
Grafana	Dashboard metrics và giao diện truy vấn log.
Loki + Promtail	Nhận và truy vấn log container.

1.5. Kiến trúc tổng thể

Trình duyệt gửi yêu cầu đến cổng 8080 của máy chủ, Nginx chuyển tiếp tới ứng dụng Flask ở mạng Docker. Ứng dụng kết nối PostgreSQL qua mạng backend nội bộ. Prometheus gọi endpoint /metrics của ứng dụng; Grafana đọc dữ liệu Prometheus và

Loki; Promtail phát hiện log Docker và gửi tới Loki. pgAdmin được cung cấp qua cổng 5050 và kết nối DB trong mạng backend.

Hình 1.1 – Sơ đồ kiến trúc: chèn hình minh họa do sinh viên tạo, thể hiện Browser → Nginx → Flask/Gunicorn → PostgreSQL; Prometheus → /metrics; Grafana → Prometheus/Loki; Promtail → Loki.

CHƯƠNG 2. PHÂN TÍCH VÀ XÂY DỰNG WEBSITE

2.1. Yêu cầu chức năng

Mã	Chức năng	Mô tả
FR-01	Xem danh sách	Hiển thị các công thức đang có.
FR-02	Tìm kiếm	Lọc theo tên món hoặc danh mục.
FR-03	Xem chi tiết	Hiển thị nguyên liệu, cách làm và thời gian.
FR-04	Thêm công thức	Nhập và lưu món ăn mới vào PostgreSQL.
FR-05	Health check	Endpoint /health kiểm tra kết nối DB.
FR-06	Metrics	Endpoint /metrics cung cấp số liệu cho Prometheus.

2.2. Thiết kế dữ liệu

Bảng recipe gồm các trường: id (khóa chính), title (tên món), category (danh mục), ingredients (nguyên liệu), instructions (cách làm) và prep_minutes (thời gian dự kiến). Trường title, category, ingredients và instructions được kiểm tra ở tầng ứng dụng; thời gian được giới hạn từ 1 đến 1440 phút.

Trường	Kiểu dữ liệu	Ý nghĩa
id	INTEGER, PK	Mã công thức.
title	VARCHAR(160)	Tên món ăn.
category	VARCHAR(80)	Danh mục món.
ingredients	TEXT	Danh sách nguyên liệu.
instructions	TEXT	Hướng dẫn chế biến.
prep_minutes	INTEGER	Thời gian dự kiến, tính bằng phút.

2.3. Giao diện và luồng hoạt động

Trang chủ hiển thị thẻ công thức, danh mục, thời gian và liên kết xem chi tiết. Ô tìm kiếm gửi tham số q qua phương thức GET. Trang chi tiết hiển thị đầy đủ dữ liệu. Trang thêm món sử dụng POST; dữ liệu được loại bỏ khoảng trắng, kiểm tra trường bắt buộc và giới hạn độ dài trước khi ghi vào cơ sở dữ liệu.

Hình 2.1 – Chụp trang chủ tại http://localhost:8080 sau khi chạy hệ thống.

Hình 2.2 – Chụp trang chi tiết một món ăn.

Hình 2.3 – Chụp biểu mẫu thêm công thức và thông báo thành công.

2.4. Tổ chức mã nguồn

Mã nguồn được chia thành các thư mục app, nginx, prometheus, grafana, loki và promtail. Dockerfile của ứng dụng sử dụng image Python slim, cài thư viện từ

requirements.txt và chạy Gunicorn bằng tài khoản hệ thống appuser có UID 10001. Cấu hình được đặt trong Git để có thể theo dõi lịch sử thay đổi.

2.5. Khởi tạo dữ liệu

Khi ứng dụng khởi động, Flask-SQLAlchemy tạo bảng nếu chưa tồn tại. Nếu bảng chưa có dữ liệu, ba công thức mẫu được thêm vào để người chấm có thể thử chức năng tìm kiếm và xem chi tiết. Cách khởi tạo này thuận tiện cho bài thực hành; với hệ thống lớn nên dùng migration có phiên bản.

CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG BẰNG DOCKER COMPOSE

3.1. Chuẩn bị môi trường

- Cài Docker Desktop trên Windows và bật WSL 2 nếu được yêu cầu.
- Mở Docker Desktop, đợi trạng thái Engine chạy ổn định.
- Giải nén mã nguồn vào một thư mục không có ký tự quá đặc biệt, ví dụ C:\De29_Recipe_Observability.
- Trong PowerShell, chuyển đến thư mục dự án và chạy docker compose config để kiểm tra cú pháp.

3.2. Khởi chạy

Chạy lệnh docker compose up -d --build. Lần đầu Docker cần tải các image nên có thể mất vài phút tùy kết nối. Sau đó chạy docker compose ps để xem trạng thái. Nếu dịch vụ chưa ổn định, dùng docker compose logs --tail=100 <tên-dịch-vụ> để tìm nguyên nhân.

3.3. Mạng và lưu trữ

Compose định nghĩa hai mạng. Mạng frontend phục vụ Nginx và các thành phần giám sát; mạng backend được khai báo internal: true để cô lập DB khỏi mạng ngoài. Dịch vụ PostgreSQL không ánh xạ cổng ra máy host, do đó chỉ các dịch vụ cùng mạng backend mới kết nối trực tiếp. Dữ liệu DB, Prometheus, Grafana và Loki lưu trong named volumes để tồn tại sau khi container được tạo lại.

3.4. Truy cập các giao diện

Dịch vụ	Địa chỉ	Mục đích
Website	http://localhost:8080	Sử dụng chức năng công thức.
pgAdmin	http://localhost:5050	Quản trị PostgreSQL.
Grafana	http://localhost:3000	Dashboard và Explore.
Prometheus	http://localhost:9090	Targets và truy vấn PromQL.
Loki API	http://localhost:3100	API log nội bộ; không cần truy cập trực tiếp thường xuyên.

3.5. Kết nối pgAdmin

Đăng nhập pgAdmin bằng email và mật khẩu trong README. Chọn Register → Server. Tab General đặt tên Recipe DB. Tab Connection điền Host name/address là db, Port 5432, Maintenance database recipes, Username recipe_user và mật khẩu trong biến POSTGRES_PASSWORD. Sau khi kết nối, mở Databases → recipes → Schemas → public → Tables để xem bảng recipe.

Hình 3.1 – Chụp màn hình docker compose ps với các dịch vụ đang Up.

Hình 3.2 – Chụp website truy cập qua Nginx tại localhost:8080.

Hình 3.3 – Chụp pgAdmin đã kết nối và hiển thị bảng recipe.

CHƯƠNG 4. REVERSE PROXY VÀ HARDENING

4.1. Nginx reverse proxy

Nginx lắng nghe cổng 80 trong container, được ánh xạ sang cổng 8080 của máy host. Khối location / chuyển tiếp yêu cầu đến http://web:8000. Các header Host, X-Real-IP, X-Forwarded-For và X-Forwarded-Proto được chuyển tiếp để ứng dụng có ngữ cảnh yêu cầu. server_tokens off hạn chế tiết lộ phiên bản Nginx.

4.2. Security headers

Header	**Tác dụng**
X-Content-Type-Options: nosniff	Ngăn trình duyệt tự suy đoán MIME type.
X-Frame-Options: SAMEORIGIN	Giảm nguy cơ nhúng trang vào frame từ nguồn khác.
Referrer-Policy	Hạn chế thông tin referrer được gửi đi.
Permissions-Policy	Tắt quyền camera, microphone và geolocation.
Content-Security-Policy	Giới hạn nguồn nội dung, form và frame theo cấu hình mẫu.

Các header là lớp bảo vệ cơ bản, không bảo đảm website an toàn tuyệt đối. Cấu hình CSP cần được kiểm thử khi bổ sung JavaScript, CSS hoặc tài nguyên ngoài.

4.3. Các biện pháp hardening

- Ứng dụng chạy bằng user không phải root trong image.
- Container web đặt read_only: true và chỉ cho phép ghi vào /tmp thông qua tmpfs.
- cap_drop: ALL loại bỏ Linux capabilities không cần thiết của web và Nginx.
- no-new-privileges:true ngăn tiến trình nâng quyền qua cơ chế setuid/setgid.
- pids_limit giới hạn số tiến trình của container web.
- PostgreSQL không công khai cổng ra host và nằm trong mạng backend nội bộ.
- Cấu hình Nginx chỉ đọc; cấu hình Prometheus, Loki và Promtail được mount read-only.
- Đăng ký tài khoản giám sát bị tắt trong Grafana; mật khẩu mẫu phải được đổi trước khi dùng ngoài máy cá nhân.

4.4. HTTPS

Đề yêu cầu HTTPS tự ký hoặc security headers cơ bản. Bộ cấu hình hiện triển khai security headers cơ bản để phù hợp môi trường local, không cấu hình HTTPS. Khi cần HTTPS, có thể bổ sung chứng chỉ và listener 443 trong Nginx hoặc đặt reverse proxy có TLS phía trước. Không nên bật HSTS khi chưa có HTTPS ổn định.

4.5. Đánh giá giới hạn

Docker socket được Promtail đọc ở chế độ chỉ đọc để phát hiện container và thu thập log. Dù mount read-only, quyền truy cập Docker socket vẫn nhạy cảm và cần được hạn chế trong môi trường production. Với triển khai thực tế nên xem xét agent

log thay thế, kiểm soát quyền, TLS, secret manager, backup và cập nhật image định kỳ.

CHƯƠNG 5. GIÁM SÁT METRICS VỚI PROMETHEUS VÀ GRAFANA

5.1. Metrics của ứng dụng

Ứng dụng xuất metrics tại /metrics thông qua prometheus-client. Bộ đếm http_requests_total ghi nhận số yêu cầu theo method, endpoint và status. Histogram http_request_duration_seconds ghi nhận thời gian xử lý. Gauge recipe_records_total thể hiện số công thức. Các metrics này giúp kiểm tra hoạt động và quan sát xu hướng, không thay thế hệ thống APM đầy đủ.

5.2. Cấu hình Prometheus

Prometheus scrape theo chu kỳ 15 giây. Job prometheus theo dõi chính Prometheus; job recipe-web lấy dữ liệu tại web:8000/metrics. Do hai dịch vụ cùng nằm trong mạng frontend, Prometheus có thể phân giải hostname Docker của ứng dụng.

5.3. Truy vấn mẫu

Mục tiêu	**PromQL**
Các target đang hoạt động	up
Target ứng dụng	up{job="recipe-web"}
Tốc độ request 5 phút	sum(rate(http_requests_total[5m]))
Tỷ lệ lỗi HTTP 5xx	sum(rate(http_requests_total{status=~"5.."}[5m]))
Độ trễ p95	histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
Tổng số công thức	recipe_records_total

5.4. Grafana dashboard

Datasource Prometheus và Loki được khai báo tự động qua provisioning. Dashboard Recipe Website Overview gồm tốc độ request, trạng thái ứng dụng, độ trễ p95, tổng số công thức và phân bố trạng thái HTTP. Truy cập Grafana, đăng nhập bằng thông tin trong README, mở Dashboards → Recipe Website Overview.

Hình 5.1 – Chụp Prometheus → Status → Targets, xác nhận recipe-web UP.

Hình 5.2 – Chụp dashboard Grafana sau khi tải trang chủ và thực hiện tìm kiếm vài lần.

5.5. Cách tạo dữ liệu kiểm tra

Mở trang chủ và tải lại nhiều lần, tìm một công thức, mở trang chi tiết và truy cập /health. Chờ ít nhất hai chu kỳ scrape rồi mở Prometheus và chạy các truy vấn. Nếu biểu đồ trống, kiểm tra trạng thái target, địa chỉ metrics và log ứng dụng.

CHƯƠNG 6. TẬP TRUNG LOG VỚI LOKI VÀ PROMTAIL

6.1. Mục tiêu

Log tập trung cho phép tìm lỗi xuyên suốt các dịch vụ mà không cần mở từng container. Ứng dụng ghi log ra stdout/stderr theo định dạng có thời gian và mức log; Docker lưu log container; Promtail phát hiện container thông qua Docker service discovery và gửi log tới Loki.

6.2. Loki và nhãn log

Loki nhận log tại API push. Promtail gắn nhãn container và service từ metadata Docker. Cấu hình nhãn giúp lọc theo dịch vụ mà không cần quét toàn bộ nội dung. Trong môi trường học tập có thể dùng nhãn {container="recipe-web"}; nhãn cụ thể cần đối chiếu với dữ liệu thực tế trong Explore.

6.3. Truy vấn LogQL

Mục tiêu	**LogQL gợi ý**
Log của ứng dụng	{container="recipe-web"}
Log Nginx	{container="recipe-nginx"}
Log có thông báo tạo công thức	{container="recipe-web"} |= "recipe_created"
Log lỗi	{container="recipe-web"} |~ "ERROR|Exception|failed"

Nếu nhãn không khớp, mở Grafana → Explore → datasource Loki, chọn Label browser và chọn nhãn container đang có. Promtail dùng Docker socket nên nếu log không xuất hiện, kiểm tra trạng thái promtail, quyền truy cập socket và log của chính Promtail.

Hình 6.1 – Chụp Grafana Explore với truy vấn LogQL có kết quả thực tế.

6.4. Phân biệt metrics và logs

Metrics là chuỗi số liệu theo thời gian, phù hợp theo dõi xu hướng và cảnh báo. Logs là sự kiện có nội dung, phù hợp điều tra nguyên nhân cụ thể. Khi có lỗi, có thể dùng dashboard phát hiện thời điểm tăng lỗi, sau đó truy vấn Loki trong khoảng thời gian tương ứng.

CHƯƠNG 7. KẾT QUẢ THỰC NGHIỆM VÀ KIỂM THỬ

Chương này cần được cập nhật bằng kết quả chạy thật trên máy tính. Bảng bên dưới là kế hoạch kiểm thử và tiêu chí mong đợi, không phải tuyên bố rằng các kiểm thử đã được thực hiện. Điền cột kết quả thực tế sau khi chạy từng bước.

Mã	Kịch bản	Cách kiểm tra	Kết quả mong đợi	Kết quả thực tế
TC-01	Kiểm tra cấu hình	docker compose config	Không có lỗi cú pháp	Điền sau khi chạy
TC-02	Trạng thái container	docker compose ps	Các dịch vụ ở trạng thái Up/healthy phù hợp	Điền sau khi chạy
TC-03	Truy cập website	Mở localhost:8080	Trang chủ hiển thị công thức mẫu	Điền sau khi chạy
TC-04	Tìm kiếm	Tìm “phở”	Danh sách lọc đúng công thức phù hợp	Điền sau khi chạy
TC-05	Thêm công thức	Gửi biểu mẫu hợp lệ	Thông báo thành công và bản ghi mới hiển thị	Điền sau khi chạy
TC-06	Kiểm tra DB	pgAdmin kết nối db	Thấy bảng recipe và dữ liệu	Điền sau khi chạy
TC-07	Health check	Mở /health	HTTP 200 và database connected	Điền sau khi chạy
TC-08	Metrics	Mở /metrics	Có http_requests_total và histogram	Điền sau khi chạy
TC-09	Prometheus scrape	Status → Targets	recipe-web ở trạng thái UP	Điền sau khi chạy
TC-10	Grafana dashboard	Mở dashboard	Panel có dữ liệu sau khi tạo request	Điền sau khi chạy
TC-11	Loki/LogQL	Explore Loki	Truy vấn trả về log container	Điền sau khi chạy
TC-12	Security headers	DevTools/ PowerShell curl -I	Có các header đã khai báo	Điền sau khi chạy

7.1. Lệnh kiểm tra HTTP

Trong PowerShell có thể dùng: Invoke-WebRequest http://localhost:8080/health; Invoke-WebRequest http://localhost:8080/metrics; Invoke-WebRequest -Method Head

http://localhost:8080/. Với một số phiên bản PowerShell, lệnh HEAD có thể hiển thị khác nhau; có thể kiểm tra header qua trình duyệt Developer Tools → Network.

7.2. Ghi nhận số liệu

Sau khi hệ thống chạy ổn định, ghi lại thời gian khởi động, số container Up, trạng thái target, số công thức trước/sau khi thêm, kết quả health check và ảnh dashboard. Không tự đặt số liệu hiệu năng nếu chưa đo. Nếu đo thời gian phản hồi, nêu rõ công cụ, số lần lặp và cách tính trung bình.

7.3. Bằng chứng cần chụp

- Docker Compose ps: toàn bộ dịch vụ và trạng thái.
- Website trang chủ, trang chi tiết và thao tác thêm công thức.
- pgAdmin: kết nối tới máy chủ db và bảng recipe.
- Prometheus Targets: job recipe-web UP.
- Grafana dashboard có dữ liệu metrics.
- Grafana Explore/Loki với ít nhất 2–3 truy vấn LogQL.
- HTTP response headers của Nginx.
- GitHub repository có README và ba commit rõ ràng.

CHƯƠNG 8. QUẢN LÝ MÃ NGUỒN GITHUB VÀ ĐỐI CHIẾU TIÊU CHÍ

8.1. Đưa dự án lên GitHub

Tạo repository công khai hoặc theo yêu cầu giảng viên, đặt tên dễ nhận diện, ví dụ DTC245200489-De29-Recipe-Deployment. Không đưa file .env, mật khẩu thật, token hoặc dữ liệu cá nhân vào Git. Đưa .env.example lên repository để mô tả các biến cấu hình.

Các lệnh khởi tạo trong thư mục dự án:

```
git init

git add .

git commit -m "feat: build recipe website and database"

git branch -M main

git remote add origin https://github.com/<username>/<repository>.git

git push -u origin main
```

Để có ba commit rõ ràng, sau commit ban đầu có thể thực hiện các thay đổi theo từng nhóm: (1) Nginx reverse proxy và security headers; (2) Prometheus/Grafana; (3) Loki/Promtail và hardening. Mỗi commit cần phản ánh thay đổi thực sự, không tạo commit giả chỉ để đủ số lượng.

8.2. Đối chiếu tiêu chí chấm

Tiêu chí	Thành phần đáp ứng	Bằng chứng cần có
Quản lý mã nguồn	Repository chứa source, Compose, README	URL repo, lịch sử commit
Ứng dụng + DB	Flask, PostgreSQL, pgAdmin	Website hoạt động, dữ liệu DB
Nginx reverse proxy	Nginx proxy_pass và security headers	Truy cập cổng 8080, header response
Prometheus + Grafana	Metrics endpoint, scrape job, dashboard	Target UP, dashboard có dữ liệu
Loki + Promtail	Docker service discovery, LogQL	Truy vấn 2–3 câu lệnh thành công
Hardening	non-root, network isolation, read-only, cap_drop, headers	Compose và kiểm tra cấu hình
Tổng thể/trình bày	README, báo cáo, demo	Báo cáo hoàn chỉnh, ảnh thật, demo trực tiếp

8.3. Những điểm cần hoàn tất trước khi nộp

- Chạy thử toàn bộ stack trên máy tính sẽ dùng để demo.
- Chụp ảnh thực tế và thay các dòng “chèn ảnh” bằng hình đã chụp.

- Điền cột kết quả thực tế trong bảng kiểm thử.
- Cập nhật mục lục trong Word bằng Ctrl+A → F9.
- Điền URL GitHub thật sau khi tạo repository.
- Kiểm tra mật khẩu mẫu, không để lộ secret cá nhân.
- In thử một vài trang để kiểm tra font, lề và ngắt trang.

KẾT LUẬN

Đề tài xây dựng một website công thức nấu ăn và mô tả triển khai nhiều dịch vụ liên kết bằng Docker Compose. Ứng dụng sử dụng PostgreSQL làm nơi lưu dữ liệu, Nginx làm reverse proxy, Prometheus thu thập metrics, Grafana trực quan hóa, còn Loki và Promtail tập trung log. Các biện pháp hardening được áp dụng ở mức cơ bản nhằm giảm quyền container và cô lập mạng cơ sở dữ liệu.

Kết quả cuối cùng cần được xác nhận bằng việc chạy thực tế trên máy sinh viên. Sau khi hoàn thiện các ảnh chụp, điền kết quả kiểm thử, cập nhật mục lục và đưa mã nguồn lên GitHub, báo cáo sẽ cung cấp được bằng chứng rõ ràng đối với các tiêu chí của đề bài.

HƯỚNG PHÁT TRIỂN

- Bổ sung đăng nhập, phân quyền và kiểm tra CSRF cho thao tác ghi dữ liệu.
- Thiết lập HTTPS với chứng chỉ phù hợp và cấu hình HSTS sau khi HTTPS ổn định.
- Quản lý secret an toàn, đổi mật khẩu mặc định và không đưa secret vào Git.
- Thêm backup/restore PostgreSQL, migration và kiểm thử khôi phục.
- Thiết lập alert rule cho lỗi HTTP, target down và độ trễ tăng cao.
- Đưa hệ thống lên máy chủ thử nghiệm có domain, giới hạn truy cập và giám sát tài nguyên host.

TÀI LIỆU THAM KHẢO

- Docker, Docker Compose documentation, https://docs.docker.com/compose/
- Flask documentation, https://flask.palletsprojects.com/
- PostgreSQL documentation, https://www.postgresql.org/docs/
- Nginx documentation, https://nginx.org/en/docs/
- Prometheus documentation, https://prometheus.io/docs/
- Grafana documentation, https://grafana.com/docs/grafana/latest/
- Loki documentation, https://grafana.com/docs/loki/latest/
- Promtail documentation, https://grafana.com/docs/loki/latest/send-data/promtail/